System Walkthrough

Domain & Problem

```python
import pandas as pd

# Loading the data using chunking to save RAM
file_path = "/content/amazon_reviews_us_Grocery_v1_00.tsv"
chunk_size = 100000
subset_list = []

# Need 10,000 high-quality rows
target_size = 10000

for chunk in pd.read_csv(file_path, sep='\t', chunksize=chunk_size, on_bad_lines='skip'):
    # Quality Filter: Only reviews with a headline AND body longer than 30 chars
    filtered = chunk[chunk['review_body'].str.len() > 30].copy()
    subset_list.append(filtered)

    # Stop once we have enough raw material to sample from
    if sum(len(c) for c in subset_list) > target_size * 5:
        break

# Combining and taking a reproducible random sample
full_potential_subset = pd.concat(subset_list)
research_corpus = full_potential_subset.sample(n=target_size, random_state=42)

# Saving it - this is the file that will be used for the project
research_corpus.to_csv("corpus.csv", index=False)
print(f"Subset created with {len(research_corpus)} documents.")
```

```
Subset created with 10000 documents.
```

This project is a **hybrid IR system** built on 10,000 Amazon Grocery product reviews sampled from a 2.4 million- record Kaggle dataset. The core problem: keyword search fails on intent-driven queries. For example, a user searching 'keto friendly electrolyte powder' misses every review that says 'low-carb' or 'sugar-free'. The system addresses this through three retrieval strategies: BM25, dense semantic search, hybrid RRF fusion.

Loading Corpus

```python
df = pd.read_csv("corpus.csv")

# Keep only the columns we need
df = df[['review_id', 'product_title', 'review_headline', 'review_body',
         'star_rating', 'product_category']].copy()

# Drop rows where review_body is missing
df.dropna(subset=['review_body'], inplace=True)
df.reset_index(drop=True, inplace=True)

print(f"Corpus size: {len(df)} documents")
df.head(2)
```

```
Corpus size: 10000 documents
```

	review_id	product_title	review_headline	review_body	star_rating	product_category
0	R1FRHVUAG8IJBE	Kirkland Signature Dried Natural Cherries 20 O...	Delicious!	These dried cherries are delicious. They are v...	5	Grocery
1	R3O86GZRVQZ3CM	Buffalo Bills Rock Candy On A Stick (individua...	Happy with package job	Nice size. Bought 2 buckets for my daughters w...	5	Grocery

System Architecture

Text Cleaning Function

```python
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords

STOPWORDS = set(stopwords.words('english'))

def clean_text(text):
    if not isinstance(text, str):
        return ""
    text = text.lower()
    text = re.sub(r'<.*?>', ' ', text)          # remove HTML tags
    text = re.sub(r'[^a-z0-9\s]', ' ', text)   # remove punctuation
    text = re.sub(r'\s+', ' ', text).strip()    # normalize whitespace
    return text

def tokenize(text):
    tokens = clean_text(text).split()
    tokens = [t for t in tokens if t not in STOPWORDS and len(t) > 1]
    return tokens
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

Two preprocessing paths:

clean_text lowercases, strips HTML, and removes punctuation; used for sentence transformers which need natural language.

tokenize additionally removes stopwords; used for BM25.

Each document combines review headline (repeated twice as signal boost) with the review body into a single field.

After filtering documents with fewer than 5 tokens, the final corpus is 9,986 documents with an average of 24 tokens each.

Building the Combined Text Field

```python
# Combining headline + body for richer retrieval signal
# Headline gets repeated to give it slightly more weight
df['combined_text'] = (
    df['review_headline'].fillna('') + ' ' +
    df['review_headline'].fillna('') + ' ' +
    df['review_body'].fillna('')
)

df['clean_text'] = df['combined_text'].apply(clean_text)
df['tokens'] = df['combined_text'].apply(tokenize)

print("Sample tokens:", df['tokens'][0][:15])
```

```
Sample tokens: ['delicious', 'delicious', 'dried', 'cherries', 'delicious', 'tender', 'juicy', 'dried', 'cherry', 'enjoy', 'found', 'dried', 'cherries', 'pits', 'stems']
```

Sanity Check

```python
print(f"Total documents:       {len(df)}")
print(f"Avg tokens per doc:    {df['tokens'].apply(len).mean():.1f}")
print(f"Docs with <5 tokens:   {df['tokens'].apply(len).lt(5).sum()}")
print(f"\nSample cleaned text:\n{df['clean_text'][0][:200]}")
```

```
Total documents:       10000
Avg tokens per doc:    24.5
Docs with <5 tokens:   14

Sample cleaned text:
delicious delicious these dried cherries are delicious they are very tender and juicy for a dried cherry which is what i enjoy about them i haven t found any dried out cherries pits or stems in the ba
```

Dropping documents that are too sparse to retrieve meaningfully

```python
df = df[df['tokens'].apply(len) >= 5].reset_index(drop=True)
print(f"Corpus after filter: {len(df)} documents")
```

```
Corpus after filter: 9986 documents
```

System Architecture cont.

Buidling and Testing BM25

```python
from rank_bm25 import BM25Okapi

# Build the index — this takes ~10-20 seconds
corpus_tokens = df['tokens'].tolist()
bm25 = BM25Okapi(corpus_tokens)

print("BM25 index built successfully.")
print(f"Vocabulary size: {len(bm25.idf):,} unique terms")
```

```
BM25 index built successfully.
Vocabulary size: 13,501 unique terms
```

BM25 Retrieval Function

```python
def bm25_search(query, top_k=10):
    query_tokens = tokenize(query)
    scores = bm25.get_scores(query_tokens)
    top_indices = np.argsort(scores)[::-1][:top_k]

    results = df.iloc[top_indices][['review_id', 'product_title',
                                    'review_headline', 'star_rating']].copy()
    results['bm25_score'] = scores[top_indices]
    results['rank'] = range(1, top_k + 1)
    return results.reset_index(drop=True)
```

Building FAISS Index

```python
embedding_dim = doc_embeddings.shape[1]  # 384

# IndexFlatIP = Inner Product (equivalent to cosine sim since we normalized)
faiss_index = faiss.IndexFlatIP(embedding_dim)
faiss_index.add(doc_embeddings)

print(f"FAISS index built successfully.")
print(f"Total vectors indexed: {faiss_index.ntotal:,}")
```

```
FAISS index built successfully.
Total vectors indexed: 9,986
```

RRF Fusion Function

```python
def hybrid_search(query, top_k=10, k=60):
    """
    Reciprocal Rank Fusion of BM25 + Dense results.
    RRF score = 1/(k + rank_bm25) + 1/(k + rank_dense)
    k=60 is the standard default from the original RRF paper.
    """

    # --- BM25 rankings ---
    query_tokens = tokenize(query)
    bm25_scores = bm25.get_scores(query_tokens)
    # Getting ALL doc rankings (not just top-k) for proper RRF
    bm25_ranked_indices = np.argsort(bm25_scores)[::-1]

    # --- Dense rankings ---
    query_embedding = model.encode(
        [query],
        normalize_embeddings=True,
        convert_to_numpy=True
    )
    dense_scores, dense_ranked_indices = faiss_index.search(query_embedding, len(df))
    dense_ranked_indices = dense_ranked_indices[0]

    # --- Building rank lookup dictionaries ---
    bm25_rank = {doc_idx: rank for rank, doc_idx in enumerate(bm25_ranked_indices)}
    dense_rank = {doc_idx: rank for rank, doc_idx in enumerate(dense_ranked_indices)}

    # --- Computing RRF score for every document ---
    all_doc_indices = set(bm25_rank.keys()) | set(dense_rank.keys())

    rrf_scores = {}
    for doc_idx in all_doc_indices:
        r_bm25  = bm25_rank.get(doc_idx, len(df))   # penalize if missing
        r_dense = dense_rank.get(doc_idx, len(df))
        rrf_scores[doc_idx] = 1/(k + r_bm25) + 1/(k + r_dense)

    # --- Sorting by RRF score and take top-k ---
    top_indices = sorted(rrf_scores, key=rrf_scores.get, reverse=True)[:top_k]

    results = df.iloc[top_indices][['review_id', 'product_title',
                                    'review_headline', 'star_rating']].copy()
    results['rrf_score']    = [rrf_scores[i] for i in top_indices]
    results['bm25_rank']    = [bm25_rank.get(i, -1) + 1 for i in top_indices]
    results['dense_rank']   = [dense_rank.get(i, -1) + 1 for i in top_indices]
    results['rank']         = range(1, top_k + 1)

    return results.reset_index(drop=True)
```

BM25 indexes 13,501 unique terms and scores documents by term frequency and inverse document frequency.

Dense retrieval encodes all 9,986 documents into 384-dimensional vectors using all-MiniLM-L6-v2, stored in a FAISS IndexFlatIP.

Hybrid search fuses both using Reciprocal Rank Fusion: RRF score = $\frac{1}{k+rank_{BM25}} + \frac{1}{k+rank_{Dense}}$, where k=60 is the standard constant from the original RRF. RRF runs over full corpus rankings, so no document is missed.

Live Queries

```
bm25_search("gluten free snacks", top_k=5)
```

	review_id	product_title	review_headline	star_rating	bm25_score	rank
0	RQSIH4L30T7QN	Nature Valley Muesli Cereal Blueberry 11 oz Box	I am gluten free but love oats. These are sadl...	5	16.221289	1
1	R382BWXVKVF6B7	Tropical Fields Crispy Coconut Rolls with Sesa...	Gluten free?	1	14.706767	2
2	R3JTYP6FV9AWCR	Nature's Path Organic Sunrise Cereal - Crunchy...	Enjoy this gluten free cereal.	5	13.858626	3
3	R2CD1TBPTGUBZN	DeLallo Gluten Free Corn and Rice, Fusilli, 12...	BEST Gluten Free Pasta I've ever tried!	5	13.630234	4
4	R1MCNQ4N4OB7LE	Glutino Gluten Free Toaster Pastry, 5 Count	gluten free & flavor free....	2	13.589042	5

BM25 scores range from 13.6 to 16.2, purely vocabulary driven. The top result scores 16.22 because the words 'gluten' and 'free' appear frequently in that review relative to the corpus.

```
dense_search("gluten free snacks", top_k=5)
```

	review_id	product_title	review_headline	star_rating	dense_score	rank
0	R2ETVXU2A0W4TA	Enjoy Life, Snack Bar, Caramel Apple, Gluten F...	Great gluten free snack/ breakfast bars !	5	0.716412	1
1	R382BWXVKVF6B7	Tropical Fields Crispy Coconut Rolls with Sesa...	Gluten free?	1	0.716029	2
2	RNRAJVB2T81JI	Pamelas Whenever Oat Bars - 4 variety pack	Being Gluten free is a lot easier with these w...	5	0.666457	3
3	R2U2L98KFNBIZ	Snyder's of Hanover Gluten Free Pretzels Multi...	Amazing GF pretzels	5	0.651395	4
4	R3SOHWT0E9MRS4	Barilla Gluten Free Spaghetti Pasta	AWESOME!	5	0.641590	5

Dense cosine similarity scores cluster tightly between 0.64-0.72, reflecting semantic proximity. Notice it surfaces 'Enjoy Life Snack Bar' at rank 1 (score 0.716) and 'Pamelas Whenever Oat Bars' at rank 3, neither of these appear in BM25's top 5 at all, because their reviews use different vocabulary.

```
hybrid_search("gluten free snacks", top_k=5)
```

	review_id	product_title	review_headline	star_rating	rrf_score	bm25_rank	dense_rank	rank
0	R382BWXVKVF6B7	Tropical Fields Crispy Coconut Rolls with Sesa...	Gluten free?	1	0.032787	2	2	1
1	RQSIH4L30T7QN	Nature Valley Muesli Cereal Blueberry 11 oz Box	I am gluten free but love oats. These are sadl...	5	0.030556	1	13	2
2	RNRAJVB2T81JI	Pamelas Whenever Oat Bars - 4 variety pack	Being Gluten free is a lot easier with these w...	5	0.030415	11	3	3
3	RDMSOFVTKGM51	Quaker Rice Cakes Apple Cinnamon 6.53 Oz - 6 U...	Okay tasting, good snack when dieting and glut...	5	0.028992	9	11	4
4	R2ETVXU2A0W4TA	Enjoy Life, Snack Bar, Caramel Apple, Gluten F...	Great gluten free snack/ breakfast bars !	5	0.028862	23	1	5

Hybrid RRF tells the most interesting story through its rank columns. 'Tropical Fields Crispy Coconut Rolls' was ranked #2 by BM25 and #2 by dense, consistent agreement from both methods pushes it to hybrid rank #1. Meanwhile 'Enjoy Life Snack Bar' had a poor BM25 rank of 23 but dense rank of 1, RRF still promotes it to hybrid rank #5, rescuing a semantically relevant result that keyword search buried. This cross-signal promotion is the core advantage of RRF fusion.

Live Queries

```
bm25_search("bulk rice for long term food storage", top_k=5)
```

	review_id	product_title	review_headline	star_rating	bm25_score	rank
0	R20UDRG3JURGO6	Rani Mustard Seeds	Five Stars	5	19.064030	1
1	R33ZLZURYT77I6	Carnation Breakfast Essentials 40-1.26oz Packe...	Five Stars	5	15.384743	2
2	R2063SRB7PBKGU	Minute Instant Enriched Long Grain	Great big boxes	5	13.733251	3
3	RXK44ISOCGP9H	Red Feather PURE CANNED BUTTER - 2 cans of 12o...	Butter is Butter	5	12.780201	4
4	R2YDO7JZII1RRT	UNCLE BEN'S Ready Rice Cheddar & Broccoli Pouch	Uncle Ben's Rice, always a favorite	5	12.201941	5

BM25 scores range from 12.2 to 19.1 but the results are poor. The top result is 'Rani Mustard Seeds' which is completely irrelevant, scoring 19.06 simply because words like 'bulk' and 'long' appeared frequently in that review. BM25 has no understanding of the actual intent behind the query.

```
dense_search("bulk rice for long term food storage", top_k=5)
```

	review_id	product_title	review_headline	star_rating	dense_score	rank
0	R25DE8AN8GG4BH	Lundberg Organic Short Grain Brown Rice, 32-Ounce	Delicious	5	0.610177	1
1	R21M2VKS4IC6M0	Organic Harvest Medley Rice Blend 3.2lbs	VERY Dense Rice	2	0.577715	2
2	R2SKZZ2UHOP1O6	Formosa Brand Pork Sung, 18 Oz	This with rice and you're ALLL good.	5	0.562345	3
3	R3HQQS3ABOC86A	Purple Rice (American Long Grain)	Phenomenal product!!	5	0.561427	4
4	R1FSVNEX3JTA4B	Tilda Legendary Rice, Pure Original Basmati, 1...	Mew	4	0.560926	5

Dense immediately corrects this as all 5 results are actual rice products with cosine scores between 0.56-0.61. It understood the semantic concept of rice storage even though the reviews do not use the phrase 'long term food storage' verbatim. 'Lundberg Organic Short Grain Brown Rice' ranks #1 at 0.610.

Query: *'bulk rice for long term food storage'* run through the interactive interface using **Hybrid + RAG mode**.
The top 5 hybrid results show the RRF fusion at work, 'Formosa Brand Pork Sung' reaches rank #1 with RRF score 0.030415 despite only having BM25 rank 11, because dense ranked it #3. 'Nozomi Super Premium Short Grain Rice 15-Pound' lands at rank #2 with BM25 rank 16 but dense rank 6, a result BM25 alone would have buried completely.
The RAG answer below demonstrates grounded generation: the model correctly identifies 'Nozomi Super Premium Short Grain Rice, 15-Pound' as the strongest recommendation based on its 5-star review and 15-pound quantity signaling bulk suitability. Critically, it also flags Document 1 (Formosa Pork Sung) as irrelevant to bulk rice, the model is reasoning over the retrieved context honestly rather than hallucinating relevance. It explicitly states when documents don't provide sufficient information, exactly the behavior the system prompt was designed to enforce.

```
from ipywidgets import widgets
from IPython.display import display, clear_output

# Input box
query_input = widgets.Text(
    placeholder="e.g. best protein bars for gym",
    description="Query:",
    layout=widgets.Layout(width="500px")
)

# Dropdown for method
method_dropdown = widgets.Dropdown(
    options=["Hybrid + RAG", "BM25 only", "Dense only", "Hybrid only"],
    value="Hybrid + RAG",
    description="Method:",
)

# Button
run_button = widgets.Button(
    description="Search",
    button_style="primary"
)

# Output area
output_area = widgets.Output()

def on_search_clicked(b):
    with output_area:
        clear_output()
        query = query_input.value.strip()
        method = method_dropdown.value

        if not query:
            print("Please enter a query.")
            return

        if method == "BM25 only":
            display(bm25_search(query, top_k=5))

        elif method == "Dense only":
            display(dense_search(query, top_k=5))

        elif method == "Hybrid only":
            display(hybrid_search(query, top_k=5))

        elif method == "Hybrid + RAG":
            print("Top 5 Hybrid Results:")
            display(hybrid_search(query, top_k=5))
            print("\nRAG Answer:")
            result = rag_answer(query, top_k=3)
            print(result["answer"])

run_button.on_click(on_search_clicked)

display(query_input, method_dropdown, run_button, output_area)
```

Query: bulk rice for long term food storage
Method: Hybrid + RAG
Search

Top 5 Hybrid Results:

	review_id	product_title	review_headline	star_rating	rrf_score	bm25_rank	dense_rank	rank
0	R2SKZZ2UHOP1O6	Formosa Brand Pork Sung, 18 Oz	This with rice and you're ALLL good.	5	0.030415	11	3	1
1	R1C4GXPKQ6YP9R	Nozomi Super Premium Short Grain Rice, 15-Pound	My everyday rice	5	0.028718	16	6	2
2	R266FN6XIEBPOC	Thai Sticky Rice (Sweet Rice) 5 Lbs	DON'T BUY!!!! It's NOT sticky rice!!	1	0.028485	7	16	3
3	R21M2VKS4IC6M0	Organic Harvest Medley Rice Blend 3.2lbs	VERY Dense Rice	2	0.028442	24	2	4
4	R2R9QALMGX8ZM8	Roland Foods Basmati Rice	Good stuff, would buy again.	5	0.027652	21	7	5

RAG Answer:
Based on the provided reviews, for bulk rice for long-term food storage, I would recommend the "Nozomi Super Premium Short Grain Rice, 15-Pound" (Document 2). This product has a 5-star review and is described as the reviewer's "everyday rice", suggesting it is a reliable and high-quality option. Additionally, the 15-pound quantity suggests it is suitable for bulk storage. The other rice products reviewed do not provide sufficient information for long-term storage or are negatively reviewed (Document 3), and Document 1 is not relevant to bulk rice as it reviews a different product (pork sung).

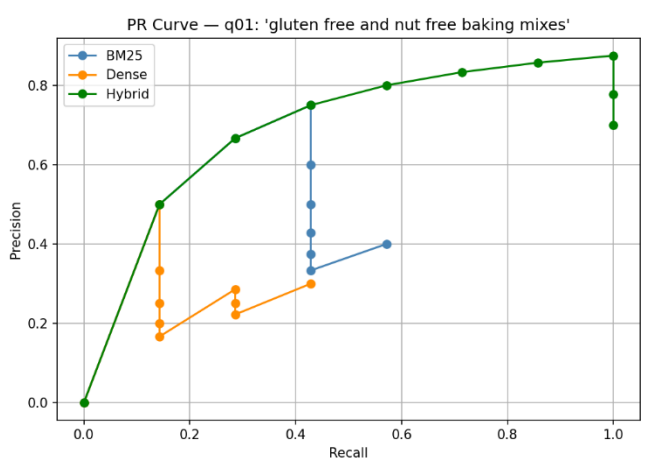
Evaluation Output & Interpretation

Computing all metrics for all methods

```python
def evaluate_run(run_df, qrels_map, method_name):
    rows = []
    for _, qrow in queries_df.iterrows():
        qid   = qrow['query_id']
        qtext = qrow['query_text']
        rel_map = qrels_map.get(qid, {})
        if not rel_map:
            continue

        ranked_ids = run_df[run_df['query_id'] == qid].sort_values('rank')['doc_id'].tolist()

        p10  = precision_at_k(ranked_ids, rel_map, k=10)
        ap_score   = average_precision(ranked_ids, rel_map)
        ndcg = ndcg_at_k(ranked_ids, rel_map, k=10)

        rows.append({
            'method':   method_name,
            'query_id': qid,
            'query_text': qtext,
            'P@10':     round(p10, 4),
            'MAP':       round(ap_score, 4),
            'nDCG@10':  round(ndcg, 4)
        })
    return pd.DataFrame(rows)

# Evaluate all 3 methods
eval_bm25   = evaluate_run(run_bm25_df,   qrels_map, 'BM25')
eval_dense  = evaluate_run(run_dense_df,  qrels_map, 'Dense')
eval_hybrid = evaluate_run(run_hybrid_df, qrels_map, 'Hybrid')

eval_all = pd.concat([eval_bm25, eval_dense, eval_hybrid], ignore_index=True)

print(eval_all.to_string(index=False))
print("\n--- Average Scores by Method ---")
print(eval_all.groupby('method')[['P@10','MAP','nDCG@10']].mean().round(4))

# Save
eval_all.to_csv("outputs/metrics_summary.csv", index=False)
print("\nSaved: outputs/metrics_summary.csv")
```

PR curve for one query

```python
import matplotlib.pyplot as plt

PR_QUERY_ID = "q01"

def pr_curve_points(ranked_doc_ids, rel_map):
    total_rel = sum(1 for v in rel_map.values() if v > 0)
    points, hits = [], 0
    for k, d in enumerate(ranked_doc_ids, start=1):
        if rel_map.get(d, 0) > 0:
            hits += 1
        precision = hits / k
        recall    = hits / total_rel if total_rel > 0 else 0
        points.append((recall, precision))
    return points

rel_map = qrels_map.get(PR_QUERY_ID, {})
qtext   = queries_df[queries_df['query_id'] == PR_QUERY_ID]['query_text'].values[0]

plt.figure(figsize=(7, 5))

for run_df, label, color in [
    (run_bm25_df,   'BM25',   'steelblue'),
    (run_dense_df,  'Dense',  'darkorange'),
    (run_hybrid_df, 'Hybrid', 'green')
]:
    ranked_ids = run_df[run_df['query_id'] == PR_QUERY_ID].sort_values('rank')['doc_id'].tolist()
    pts = pr_curve_points(ranked_ids, rel_map)
    recalls    = [p[0] for p in pts]
    precisions = [p[1] for p in pts]
    plt.plot(recalls, precisions, marker='o', label=label, color=color)

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title(f"PR Curve — {PR_QUERY_ID}: '{qtext}'")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("outputs/pr_curve.png", dpi=150)
plt.show()
print("Saved: outputs/pr_curve.png")
```




```
method query_id                                      query_text  P@10    MAP  nDCG@10
  BM25      q01                gluten free and nut free baking mixes   0.4 0.3310   0.4803
  BM25      q02            healthy low sodium snacks for weight loss   0.4 0.2374   0.4445
  BM25      q03                  best green tea for energy and focus   0.2 0.1167   0.3098
  BM25      q04           organic coconut oil for skin and hair care   0.7 0.5578   0.7242
  BM25      q05       stale popcorn and poor quality food complaints   0.0 0.0000   0.0000
  BM25      q06                sugar free peanut butter protein bars   0.2 0.3333   0.3391
  BM25      q07                 bulk rice for long term food storage   0.2 0.0607   0.1617
  BM25      q08 variety candy bags for halloween trick or treating   0.4 0.4021   0.5538
  BM25      q09           easy to make cold brew iced tea for summer   0.6 0.4573   0.5871
  BM25      q10            soy sauce alternatives like liquid aminos   0.3 0.6667   0.6714
  BM25      q11     keto friendly electrolyte powder for dehydration   0.0 0.0000   0.0000
  BM25      q12              spicy korean ramen with high heat level   0.2 0.0733   0.2342
  BM25      q13     natural sweeteners that don't have an aftertaste   0.2 0.1083   0.2440
  BM25      q14         caffeinated chocolate covered espresso beans   0.0 0.0000   0.0000
  BM25      q15                    dairy free milk powder for baking   0.2 0.2407   0.4955
  BM25      q16            high protein snacks for toddler lunch box   0.2 0.1190   0.2167
  BM25      q17                artisanal olive oil for dipping bread   0.6 0.8095   0.8255
  BM25      q18         fast acting energy gels for marathon running   0.1 0.0357   0.1184
  BM25      q19            organic raw honey with honeycomb included   0.6 0.5506   0.7573
  BM25      q20       sustainable and fair trade medium roast coffee   0.4 0.3247   0.6520
 Dense      q01                gluten free and nut free baking mixes   0.3 0.1551   0.3251
 Dense      q02            healthy low sodium snacks for weight loss   0.2 0.1667   0.2146
 Dense      q03                  best green tea for energy and focus   0.2 0.4000   0.6422
 Dense      q04           organic coconut oil for skin and hair care   0.6 0.5667   0.7152
 Dense      q05       stale popcorn and poor quality food complaints   0.1 1.0000   1.0000
 Dense      q06                sugar free peanut butter protein bars   0.1 0.0833   0.1376
 Dense      q07                 bulk rice for long term food storage   0.3 0.1577   0.3542
 Dense      q08 variety candy bags for halloween trick or treating   0.2 0.1562   0.2148
 Dense      q09           easy to make cold brew iced tea for summer   0.3 0.2135   0.4404
 Dense      q10            soy sauce alternatives like liquid aminos   0.3 0.4429   0.5551
 Dense      q11     keto friendly electrolyte powder for dehydration   0.3 0.2103   0.3068
 Dense      q12              spicy korean ramen with high heat level   0.5 0.8762   0.9092
 Dense      q13     natural sweeteners that don't have an aftertaste   0.3 0.2643   0.3963
 Dense      q14         caffeinated chocolate covered espresso beans   0.0 0.0000   0.0000
 Dense      q15                    dairy free milk powder for baking   0.1 0.3333   0.5317
 Dense      q16            high protein snacks for toddler lunch box   0.4 0.3667   0.6921
 Dense      q17                artisanal olive oil for dipping bread   0.4 0.4041   0.6403
 Dense      q18         fast acting energy gels for marathon running   0.2 0.1786   0.3616
 Dense      q19            organic raw honey with honeycomb included   0.6 0.5108   0.6584
 Dense      q20       sustainable and fair trade medium roast coffee   0.3 0.3452   0.3809
Hybrid      q01                gluten free and nut free baking mixes   0.7 0.7546   0.7403
Hybrid      q02            healthy low sodium snacks for weight loss   0.7 0.9087   0.8723
Hybrid      q03                  best green tea for energy and focus   0.5 0.6343   0.8276
Hybrid      q04           organic coconut oil for skin and hair care   1.0 1.0000   1.0000
Hybrid      q05       stale popcorn and poor quality food complaints   0.1 0.1000   0.2891
Hybrid      q06                sugar free peanut butter protein bars   0.3 0.5778   0.6352
Hybrid      q07                 bulk rice for long term food storage   0.8 0.7603   0.7921
Hybrid      q08 variety candy bags for halloween trick or treating   0.8 0.9472   0.8944
Hybrid      q09           easy to make cold brew iced tea for summer   0.8 0.8073   0.9195
Hybrid      q10            soy sauce alternatives like liquid aminos   0.3 0.6429   0.6699
Hybrid      q11     keto friendly electrolyte powder for dehydration   0.6 0.6569   0.6829
Hybrid      q12              spicy korean ramen with high heat level   0.5 0.5454   0.6955
Hybrid      q13     natural sweeteners that don't have an aftertaste   0.6 0.7861   0.8663
Hybrid      q14         caffeinated chocolate covered espresso beans   0.0 0.0000   0.0000
Hybrid      q15                    dairy free milk powder for baking   0.3 0.5397   0.7889
Hybrid      q16            high protein snacks for toddler lunch box   0.7 0.6441   0.7216
Hybrid      q17                artisanal olive oil for dipping bread   0.7 0.8806   0.9361
Hybrid      q18         fast acting energy gels for marathon running   0.7 0.6211   0.7499
Hybrid      q19            organic raw honey with honeycomb included   0.9 0.9060   0.9588
Hybrid      q20       sustainable and fair trade medium roast coffee   0.7 0.8598   0.8964

--- Average Scores by Method ---
         P@10     MAP  nDCG@10
method
BM25    0.295  0.2712   0.3908
Dense   0.285  0.3416   0.4738
Hybrid  0.585  0.6786   0.7468

Saved: outputs/metrics_summary.csv
```

Evaluation across 20 queries with 200 graded relevance judgments (0/1/2). Hybrid achieves nDCG@10 of 0.7468, a 91% improvement over BM25 (0.3908) and 57% over dense (0.4738). BM25 and dense have nearly identical P@10, meaning they surface a similar number of relevant documents in the top 10, but dense ranks them higher, hence the better nDCG. The PR curve for q01 confirms hybrid maintains the highest precision at every recall level. One honest limitation: all 200 judgments were made by a single annotator, introducing subjectivity that a multi-annotator study with inter-annotator agreement scoring would address.